

Final Project

This notebook is adapted from here: https://aiqm.github.io/torchani/examples/nnp_training.html

Checkpoint 1: Data preparation

1. Create a working directory: `/global/scratch/users/[USER_NAME]/[DIR_NAME]` . Replace the [USER_NAME] with yours and specify a [DIR_NAME] you like.
2. Copy the Jupyter Notebook to the working directory
3. Download the ANI dataset `ani_dataset_gdb_s01_to_s04.h5` from bCourses and upload it to the working directory

```
In [1]: import warnings
warnings.filterwarnings("ignore", category=UserWarning)
import numpy as np
from tqdm import tqdm
import torch
import torch.nn as nn
import torchani
```

Use GPU

```
In [2]: device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(device)
```

cpu

Set up AEV computer

AEV: Atomic Environment Vector (atomic features)

Ref: Chem. Sci., 2017, 8, 3192

```
In [3]: def init_aev_computer():
    Rcr = 5.2
    Rca = 3.5
    EtaR = torch.tensor([16], dtype=torch.float, device=device)
    ShfR = torch.tensor([
```

```

    0.900000, 1.168750, 1.437500, 1.706250,
    1.975000, 2.243750, 2.512500, 2.781250,
    3.050000, 3.318750, 3.587500, 3.856250,
    4.125000, 4.393750, 4.662500, 4.931250
], dtype=torch.float, device=device)

EtaA = torch.tensor([8], dtype=torch.float, device=device)
Zeta = torch.tensor([32], dtype=torch.float, device=device)
ShfA = torch.tensor([0.90, 1.55, 2.20, 2.85], dtype=torch.float, device=device)
ShfZ = torch.tensor([
    0.19634954, 0.58904862, 0.9817477, 1.37444680,
    1.76714590, 2.15984490, 2.5525440, 2.94524300
], dtype=torch.float, device=device)

num_species = 4
aev_computer = torchani.AEVComputer(
    Rcr, Rca, EtaR, ShfR, EtaA, Zeta, ShfA, ShfZ, num_species
)
return aev_computer

aev_computer = init_aev_computer()
aev_dim = aev_computer.aev_length
print(aev_dim)

```

384

Prepare dataset & split

```

In [6]: def load_ani_dataset(dspath):
    self_energies = torch.tensor([
        0.500607632585, -37.8302333826,
        -54.5680045287, -75.0362229210
    ], dtype=torch.float, device=device)
    energy_shifter = torchani.utils.EnergyShifter(None)
    species_order = ['H', 'C', 'N', 'O']

    dataset = torchani.data.load(dspath)
    dataset = dataset.subtract_self_energies(energy_shifter, species_order)
    dataset = dataset.species_to_indices(species_order)
    dataset = dataset.shuffle()
    return dataset

dataset = load_ani_dataset("./ani_gdb_s01_to_s04.h5")

```

```
# Use dataset.split method to do split
train_data, val_data, test_data = dataset.split(0.8, 0.1, 0.1)
```

Batching

```
In [7]: batch_size = 8192
# use dataset.collate(...).cache() method to do batching
train_data_loader = train_data.collate(batch_size).cache()
val_data_loader = val_data.collate(batch_size).cache()
test_data_loader = test_data.collate(batch_size).cache()
```

Torchani API

```
In [11]: class AtomicNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(384, 128),
            nn.ReLU(),
            nn.Linear(128, 1)
        )

    def forward(self, x):
        return self.layers(x)

net_H = AtomicNet()
net_C = AtomicNet()
net_N = AtomicNet()
net_O = AtomicNet()

# ANI model requires a network for each atom type
# use torch.ANIModel() to compile atomic networks
ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)
```

```
In [12]: train_data_batch = next(iter(train_data_loader))

loss_func = nn.MSELoss()
species = train_data_batch['species'].to(device)
coords = train_data_batch['coordinates'].to(device)
```

```
true_energies = train_data_batch['energies'].to(device).float()  
_, pred_energies = model((species, coords))  
loss = loss_func(true_energies, pred_energies)  
print(loss)
```

```
tensor(0.0494, grad_fn=<MseLossBackward0>)
```

In []: